MLT- Lab 2

Title:

PCA and K-Mean analysis on 2 unlabelled numerical attribute dataset.

Dataset Descriptiom:

- Both data sets have no missing values
- Dataset 1:
  - Size & Structure: 1000 rows, 15 columns.
  - Attributes: All 15 columns are named 'feature_1' to 'feature_15'.
  - Data Types: All features are of type float64.
  - Distribution: The features are numerically distributed around a mean close to 0, with standard deviations around 1, indicating a standard normal-like distribution (as they were generated using np.random.randn).
- Dataset 2:
  - Size & Structure: 1200 rows, 15 columns.
  - Attributes: All 15 columns are named 'feature_1' to 'feature_15'.
  - Data Types: All features are of type float64.
  - Distribution: Similar to Dataset 1, features are numerically distributed around a mean close to 0, with standard deviations around 1, consistent with a standard normal-like distribution.

Pseudo code:

1) Algorithm: PCA (Principal Component Analysis)

Input: X (dataset with n_samples rows and n_features columns), n_components (desired dimensions)

Output: X_transformed (reduced dataset with n_samples rows and n_components columns)

Step 1: Center the data

Calculate the mean of each feature across all samples

Subtract the mean from each data point to get centered data

(This ensures the data has zero mean)

Step 2: Compute the covariance matrix

Calculate how each feature varies with respect to other features

This gives us an n_features × n_features matrix

Step 3: Find eigenvalues and eigenvectors
Decompose the covariance matrix to get:
- Eigenvalues: Numbers indicating importance of each direction
- Eigenvectors: Directions of maximum variance in the data

Step 4: Sort components by importance
Arrange eigenvalues in descending order (largest to smallest)
Rearrange corresponding eigenvectors to match this order
(Larger eigenvalues = more important directions)

Step 5: Select top principal components
Keep only the first n_components eigenvectors
These represent the most important directions in the data

Step 6: Transform the data
Project the centered data onto the selected principal components
This gives us the reduced dimensionality representation

Step 7: Calculate explained variance
Compute how much variance each component captures
Report total variance explained as a percentage

Return the transformed data

2)Algorithm: K-Means Clustering
Input: X (dataset with n_samples rows and n_features columns),
k (number of clusters),
distance_metric ('euclidean' or 'mahalanobis'),
max_iterations (maximum loops allowed)
Output: labels (cluster assignment for each point),
centroids (final cluster centers),
inertia (sum of squared distances)

Step 1: Initialize
If using Mahalanobis distance:
Calculate covariance matrix of the data
Compute its inverse for distance calculations

Randomly select k data points as initial cluster centroids

```
Step 2: Assignment Phase (repeat until convergence)
  FOR each data point:
    Calculate distance to all k centroids using chosen metric:
      - Euclidean: straight-line distance
      - Mahalanobis: distance accounting for correlations
    Assign the point to the nearest centroid's cluster
  END FOR

Step 3: Update Phase
  FOR each cluster:
    Find all points assigned to this cluster
    IF cluster has at least one point:
      Calculate new centroid as the mean of all points in cluster
    ELSE:
      Keep the old centroid (empty cluster case)
    END IF
  END FOR

Step 4: Check Convergence
  IF centroids didn't change (or changed very little):
    Stop iteration - algorithm has converged
  ELSE IF reached max_iterations:
    Stop iteration - maximum attempts reached
  ELSE:
    Go back to Step 2 with updated centroids
  END IF

Step 5: Calculate Inertia
  Initialize inertia to zero
  FOR each data point:
    Calculate squared distance to its assigned centroid
    Add this to inertia
  END FOR
  (Lower inertia = tighter, more compact clusters)

Return cluster labels, final centroids, and inertia value
```

3)Function: Euclidean Distance

Input: point1, point2 (two data points)
Output: distance (scalar value)

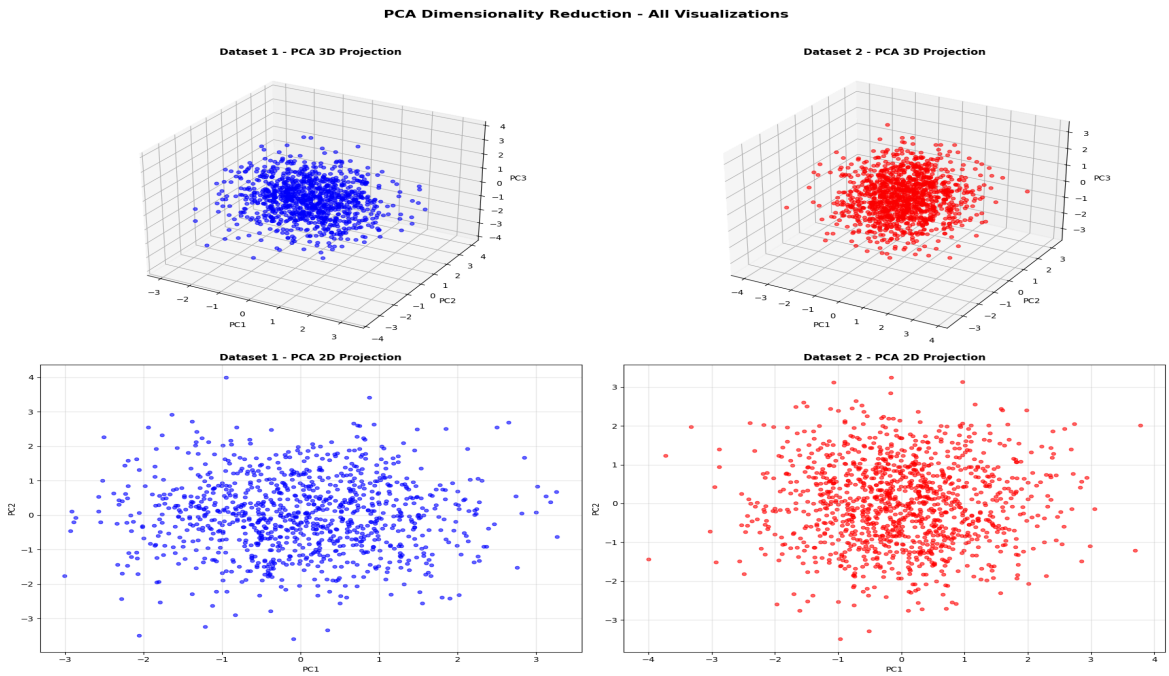
Calculate the difference between corresponding coordinates
Square each difference
Sum all squared differences
Take the square root of the sum
Return distance

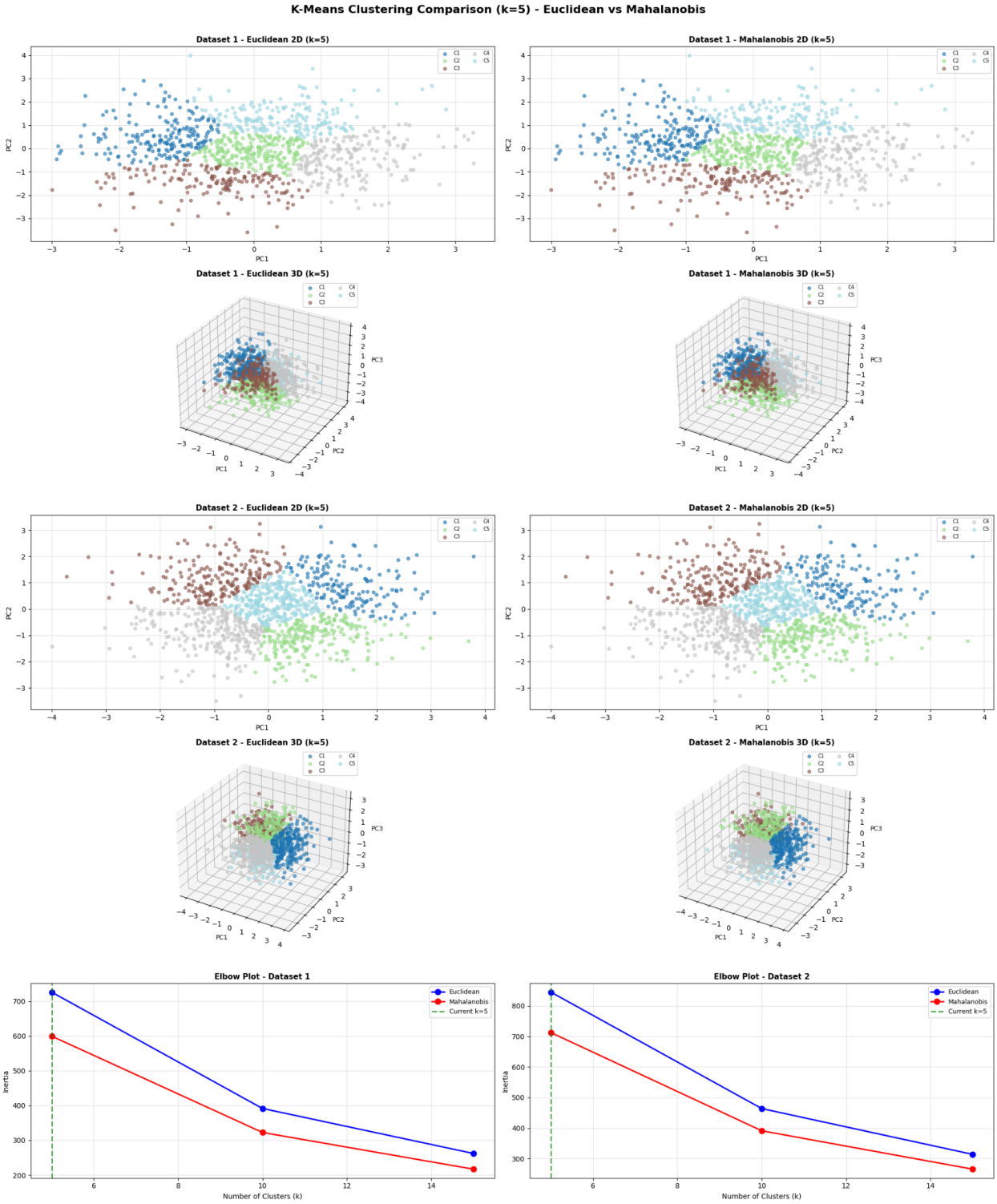
4)Function: Mahalanobis Distance

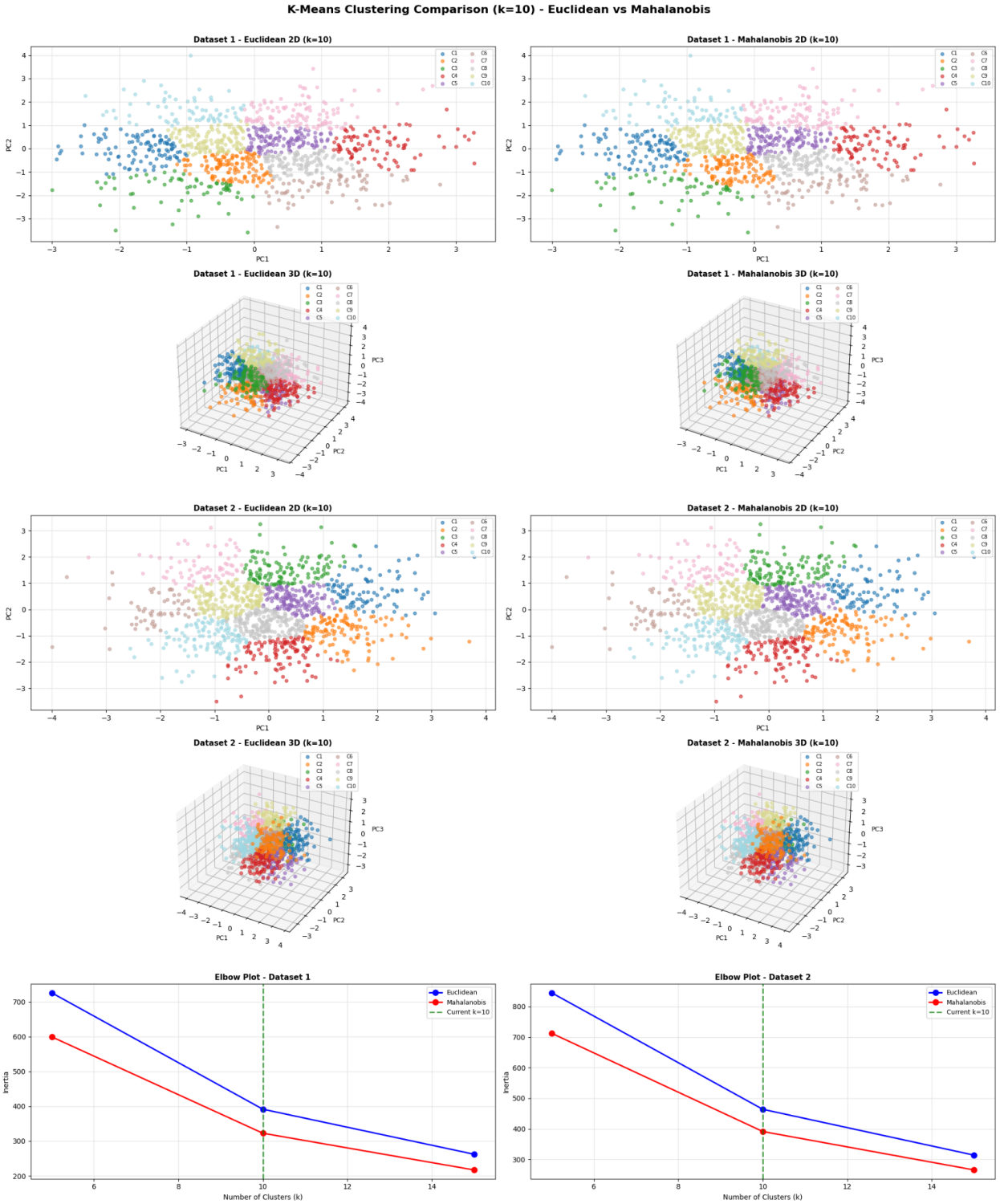
Input: point1, point2 (two data points), cov_inv (inverse covariance matrix)
Output: distance (scalar value)

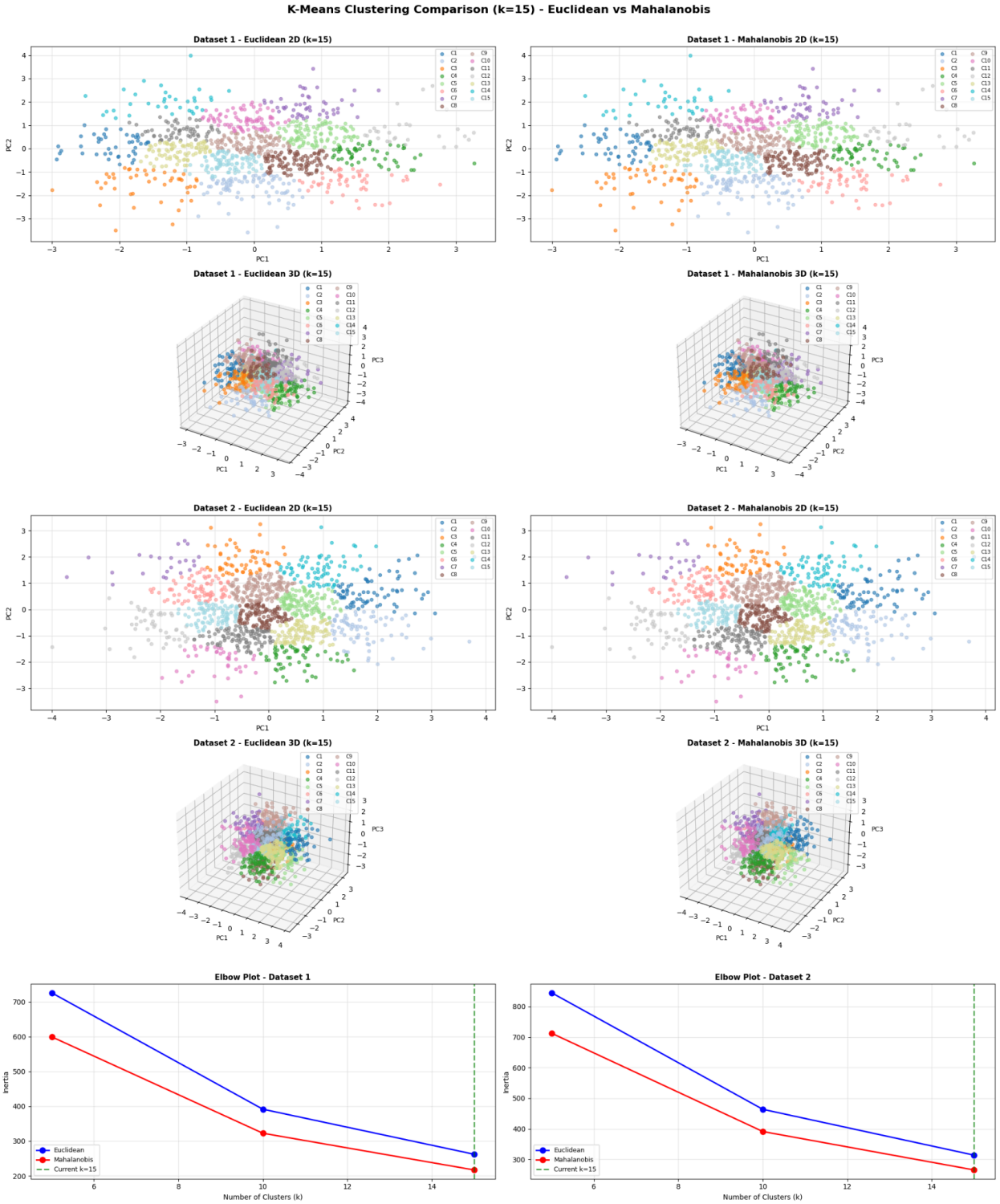
Calculate the difference vector between the two points
Multiply: difference × inverse_covariance × difference_transpose
Take the square root of the result
Return distance

Output:




K-Means Clustering Comparison (k=5) - Euclidean vs Mahalanobis
Dataset 1 - Euclidean 2D (k=5)
Dataset 1 - Mahalanobis 2D (k=5)
Dataset 1 - Euclidean 3D (k=5)
Dataset 1 - Mahalanobis 3D (k=5)
Dataset 2 - Euclidean 2D (k=5)
Dataset 2 - Mahalanobis 2D (k=5)
Dataset 2 - Euclidean 3D (k=5)
Dataset 2 - Mahalanobis 3D (k=5)
Elbow Plot - Dataset 1
Elbow Plot - Dataset 2
Euclidean
Mahalanobis
Current k=5
Inertia
Number of Clusters (k)
PC1
PC2
PC3


K-Means Clustering Comparison (k=10) - Euclidean vs Mahalanobis
Dataset 1 - Euclidean 2D (k=10)
Dataset 1 - Mahalanobis 2D (k=10)
Dataset 1 - Euclidean 3D (k=10)
Dataset 1 - Mahalanobis 3D (k=10)
Dataset 2 - Euclidean 2D (k=10)
Dataset 2 - Mahalanobis 2D (k=10)
Dataset 2 - Euclidean 3D (k=10)
Dataset 2 - Mahalanobis 3D (k=10)
Elbow Plot - Dataset 1
Elbow Plot - Dataset 2
Euclidean
Mahalanobis
Current k=10
Inertia
Number of Clusters (k)
PC1
PC2
PC3


K-Means Clustering Comparison (k=15) - Euclidean vs Mahalanobis
Dataset 1 - Euclidean 2D (k=15)
Dataset 1 - Mahalanobis 2D (k=15)
Dataset 1 - Euclidean 3D (k=15)
Dataset 1 - Mahalanobis 3D (k=15)
Dataset 2 - Euclidean 2D (k=15)
Dataset 2 - Mahalanobis 2D (k=15)
Dataset 2 - Euclidean 3D (k=15)
Dataset 2 - Mahalanobis 3D (k=15)
Elbow Plot - Dataset 1
Elbow Plot - Dataset 2
PC1
PC2
PC3
Inertia
Number of Clusters (k)
Euclidean
Mahalanobis
Current k=15

Observation:

- **Analyze PCA Results**: Summarize the PCA dimensionality reduction results for both dataset1 and dataset2. This includes discussing the explained variance ratio for different numbers of components (10, 3, 2) and observing any differences or similarities in how dimensions are reduced across the datasets. Also comment on the visualizations generated for 2D and 3D PCA projections.
- **Analyze K-Means Clustering Results - Distance Metrics**: Compare the clustering results obtained using Euclidean and Mahalanobis distances for both datasets across different 'k' values and dimensions. Discuss the impact of the distance metric on inertia values and the visual separation of clusters.
- **Analyze K-Means Clustering Results - Impact of 'k' Value**: Examine how increasing the number of clusters (k=5, 10, 15) affects the inertia and the visual appearance of clusters for both datasets and both distance metrics. Discuss the 'elbow' plots for each dataset and how they might suggest an optimal 'k'.
- **Analyze K-Means Clustering Results - Impact of Dimension**: Discuss how the choice of dimensions (2D vs. 3D PCA reduced data) impacts the K-Means clustering results for both datasets and both distance metrics. Observe if higher dimensions lead to better separation or different cluster structures, and how this relates to the explained variance of the PCA components.
- **Overall Summary of PCA and K-Means**: Provide a comprehensive summary that synthesizes the findings from PCA and K-Means clustering. This includes drawing conclusions about the effectiveness of PCA in reducing dimensionality for these datasets, and the performance of K-Means with different distance metrics, 'k' values, and dimensions.